CH-8 PART - 1

In-Depth Study Notes: Oracle SQL Set Operators and Compound Statements

As an expert Oracle SQL Developer, I have synthesized the execution logs provided in your latest files (CH8-1.TXT and CH8-2.TXT). When business requirements demand comparing, combining, or finding the differences between entirely separate datasets, standard joins are not always applicable. These notes detail the architecture of **Set Operators**, **Compound Precedence**, and **Matrix Logic Translation**.

Concept 1: Vertical Data Combination (Set Operators)

Core Purpose: Unlike JOINs, which combine tables horizontally (adding columns side-by-side), **Set Operators** combine result sets *vertically* (stacking rows on top of each other) 1. When two or more SELECT statements are joined using a set operator, the entire script is known as a **Compound Statement** 1.

Keyword Breakdown:

- **UNION ALL**: Simply clubs both result sets vertically, maintaining the exact top-to-down random order of rows, and **keeps all duplicate rows** exactly as they are 1.
- **UNION**: Clubs the result sets together, but performs two implicit overhead tasks: **1) Removes duplicate rows, and 2) Sorts the final result set in ascending order** 2.
- **INTERSECT**: Extracts only the rows that are **common** to both result sets, then removes duplicates and sorts the output in ascending order 3.
- **MINUS**: Acts as a subtractor. It completely removes any common rows from the **first** result set, and outputs only the remaining unique rows from the top query, sorted in ascending order 4.

Comparison of Set Operators:

Operator,Deduplicates Data?,Sorts Output?,Execution Speed,Business Use Case
UNION ALL,No,No,Fastest,Combining massive archives where duplicates are allowed.
UNION,Yes,Yes,Slower (CPU Sort),Generating a distinct master list from multiple tables.
INTERSECT,Yes,Yes,Slower,Finding employees who exist in both the 1998 and 1999 hiring cohorts 5.
MINUS,Yes,Yes,Slower,Finding employees in Dept 10 who do not share a Job ID with Dept 20 4.

Syntax Breakdown:

```
SELECT column_1, column_2 FROM table_a
[ SET OPERATOR ]
SELECT column_1, column_2 FROM table_b;
```

Detailed Example: The HR manager needs a report showing all unique DEPARTMENT_IDs for which the company pays salaries of $6,000 or more, explicitly excluding any departments that *also* pay salaries strictly under $6,000.

```
-- Query 1 retrieves all departments paying high salaries
```

```
SELECT DEPARTMENT_ID FROM EMPLOYEES WHERE SALARY >= 6000
MINUS
-- Query 2 subtracts departments paying low salaries
SELECT DEPARTMENT_ID FROM EMPLOYEES WHERE SALARY < 6000;
```

[Execution: Outputs only the distinct departments that exclusively pay 6000 or more 6.]

Execution Logic:

1. **Independent Evaluation**: The Oracle Engine executes the top query and the bottom query completely independently, caching their result sets in the PGA (Program Global Area).
2. **Validation**: The engine verifies that both queries have the **exact same number of columns** and that the corresponding columns have **compatible data types**.
3. **Set Operations**: If using UNION/INTERSECT/MINUS, the CPU performs an aggressive hash or sort operation to identify duplicates and order the final output before projecting it to the user.

Pro-Tips/Best Practices:

- **Always default to UNION ALL**: Sorting is one of the most expensive operations for a database CPU. If you know your data sets do not contain duplicates (e.g., combining JAN_SALES and FEB_SALES), use UNION ALL. Using UNION will force an unnecessary Full Table Sort, destroying query performance.
- **Avoiding SELECT ***: Never use SELECT * in compound statements. If a DBA adds a column to table_a but not table_b, the column counts will mismatch, and the script will instantly crash with an **ORA-01789: query block has incorrect number of result columns**. Always list columns explicitly.

Concept 2: Compound Statement Precedence

Core Purpose: When chaining three or more Set Operators together, the execution order dictates the final output. Without explicit guidance, Oracle evaluates Set Operators strictly from **top-left to down-right** 7.

Keyword Breakdown:

- **() (Parentheses)**: Used to override the default top-to-bottom execution precedence, forcing the engine to evaluate specific SELECT blocks first 7.

Comparison of Precedence Behaviors:

Query Structure,First Action,Second Action,Final Row Count (Example Data)
10 INTERSECT 20 UNION 30,Intersects 10 & 20 first.,Unions the result with 30.,3 Rows 8
10 INTERSECT (20 UNION 30),Unions 20 & 30 first.,Intersects 10 with that combined result.,2 Rows 8

Syntax Breakdown:

```
SELECT ... FROM ...
SET OPERATOR
(
    SELECT ... FROM ...
    SET OPERATOR
    SELECT ... FROM ...
);
```

Detailed Example: We want to combine Department 20 and 30 into a single master pool first, and then find which Job IDs in Department 10 do *not* exist in that combined pool.

```
SELECT JOB FROM MPSEMP WHERE DEPTNO = 10
MINUS
(
    SELECT JOB FROM MPSEMP WHERE DEPTNO = 20
    UNION 30
    SELECT JOB FROM MPSEMP WHERE DEPTNO = 30
);
```

[Execution: The engine first builds the deduplicated list of jobs from Dept 20 and 30, then subtracts those jobs from the Dept 10 list 9.]

Pro-Tips/Best Practices:

- **Always use parentheses for chained sets:** Just like mathematical operations (AND/OR), writing chained Set Operators without parentheses makes the business logic incredibly difficult for other developers to read and debug.

Concept 3: Generating Matrix Reports (Set Operators vs. Subqueries)

Core Purpose: Often, businesses want to map a Cartesian grid (e.g., "Show me every Teacher matched with every Subject, and put a YES or NO next to whether they can teach it"). While this can be done by chaining complex CROSS JOINs with MINUS and UNION ALL, modern Oracle SQL allows us to bypass heavy set operators using highly efficient Conditional Functions.

Keyword Breakdown:

- **CROSS JOIN**: Creates a Cartesian Product (every row in Table A mapped against every row in Table B).
- **CASE WHEN ... IN (Subquery)**: Evaluates multi-column logic dynamically against a subquery result.

Comparison of Matrix Mapping:

Logic Strategy,Mechanism,Performance / Readability
Set Operator Chain,CROSS JOIN -> MINUS -> UNION,"Highly verbose, forces the database to scan the tables multiple times 10."
CASE Evaluation,CROSS JOIN -> CASE WHEN IN (),Best Practice. Requires only a single pass and is highly readable 11.

Syntax Breakdown (CASE Subquery Method):

```
SELECT A.col, B.col,
       CASE
           WHEN (A.col, B.col) IN (SELECT col1, col2 FROM
mapping_table) THEN 'YES'
           ELSE 'NO'
       END AS Status_Flag
FROM table_A A CROSS JOIN table_B B;
```

Detailed Example: An institute needs a master grid of all staff and all subjects (MPSSTAFF x MPSSUBJECT), verifying their abilities against the MPSCANTEACH table.

```
SELECT T.NAME AS STAFF, B.NAME AS SUBJECT,
       CASE
```

```
           WHEN (T.NAME, B.NAME) IN (SELECT STAFF, SUBJECT FROM
MPSCANTEACH)
           THEN 'YES'
           ELSE 'NO'
       END AS "CAN_TEACH?"
FROM MPSSTAFF T CROSS JOIN MPSSUBJECT B;
```

[Execution: Generates all 12 possible combinations, checks each tuple (ASHWIN, TOEFL) against the MPSCANTEACH table, and efficiently assigns the YES/NO string 11.]